

Chapter 5

OMAP - Application Prozessor

5.1 General Aspects

- OMAP-Application Prozessor provides a lot interfaces (like simple digital I/O but also complex systems as a camera interface)
- different interfaces are a key feature of every microcontroller system
- every interface is controlled by one or more registers, which are part of the standard memory space.
- most of the interfaces can generate an interrupt (which is handled according to an interrupt service routine and can be used to program some actions).
- every interface also has a flag which makes it possible to write programs to ask in a “polling method” for interaction with the interface.
- to reduce the number of external pins, mostly the input/output pins can be used in more than one mode.
- this means the number of interfaces are also reduced.

5.2 Digital Ports

- the OMAP-Application Prozessor has a lot of digital input/output lines (interfaces).
- these lines are grouped in 6 ports, every port with 32 input/output lines.
- every port is programmable as an input or an output port.
- a mix of input and output lines in one port is allowed.
- the output lines are driven a Hiz-mode (high impedance mode, this means a connection of two or more outputs lines are possible).
- 32 bit Ports (Programming direction)

Port	Port lines	Direction
GPIO1	[31:0] (i/o)	0x48310034 (r/w)
GPIO2	[63:34] (i/o)	0x49050034 (r/w)
GPIO3	[95:64] (i/o)	0x49052034 (r/w)
GPIO4	[127:96] (i/o)	0x49054034 (r/w)
GPIO5	[159:128] (i/o)	0x49056034 (r/w)
GPIO6	[191:160] (i/o)	0x49058034 (r/w)

- after reset the direction is set to input (0xffffffff in the corresponding direction register)
- a bit reset on the correspondig bit in the direction register switch the direction to output
- 32 bit Ports (Read and write access)

Port	Port lines	Input	Output
GPIO1	[31:0] (i/o)	0x48310038 (r)	0x4831003c (r/w)
GPIO2	[63:34] (i/o)	0x49050038 (r)	0x4905003c (r/w)
GPIO3	[95:64] (i/o)	0x49052038 (r)	0x4905203c (r/w)
GPIO4	[127:96] (i/o)	0x49054038 (r)	0x4905403c (r/w)
GPIO5	[159:128] (i/o)	0x49056038 (r)	0x4905603c (r/w)
GPIO6	[191:160] (i/o)	0x49058038 (r)	0x4905803c (r/w)

- read access from input lines done as standard 32 bit access
- write access to output lines could be done as standard 32 bit write access
- a read access from the output lines gets the “old” contents (shadow register)
- with the registers “ClearDataOut” and “SetDataOut” an specific Clearbit- and Setbit-Operation is possible
- the specific bit which should be changed must be set in the correspondig “ClearDataOut” or SetDataOut“ register

Port	Port lines	ClearDataOut	SetDataOut
GPIO1	[31:0] (i/o)	0x48310090 (r/w)	0x48310094 (r/w)
GPIO2	[63:34] (i/o)	0x49050090 (r/w)	0x49050094 (r/w)
GPIO3	[95:64] (i/o)	0x49052090 (r/w)	0x49052094 (r/w)
GPIO4	[127:96] (i/o)	0x49054090 (r/w)	0x49054094 (r/w)
GPIO5	[159:128] (i/o)	0x49056090 (r/w)	0x49056094 (r/w)
GPIO6	[191:160] (i/o)	0x49058090 (r/w)	0x49058094 (r/w)

5.3 Serial Ports

- The OMAP-Application Prozessor supports different kinds of serial interfaces:
 - programmable serial controller (UART) for standard asynchronus serial communication.
 - universal serial bus (USB) for serial bus-oriented communication.
 - inter-integrated circuit communication (I²C).

UART universial asynchron receiver/transmitter

- 3 different UART-Channels:
 - UART 1, 2 asynchronous data transfer with 5,6,7,8 bits of data plus parity transfer.
 - UART 3 asynchronouns data transfer IrDA
 - 1.0 SIR mode (SIR) with a baudrate in the range of 2400 - 115200 bps.
 - IrDA 1.1 MIR mode (MIR) with a baudrate ranging from 0.576 or 1.152 Mbps.

- IrDA 1.1 FIR mode (FIR) with a baudrate of 4.0 Mbps.
- programmable baudrate generator

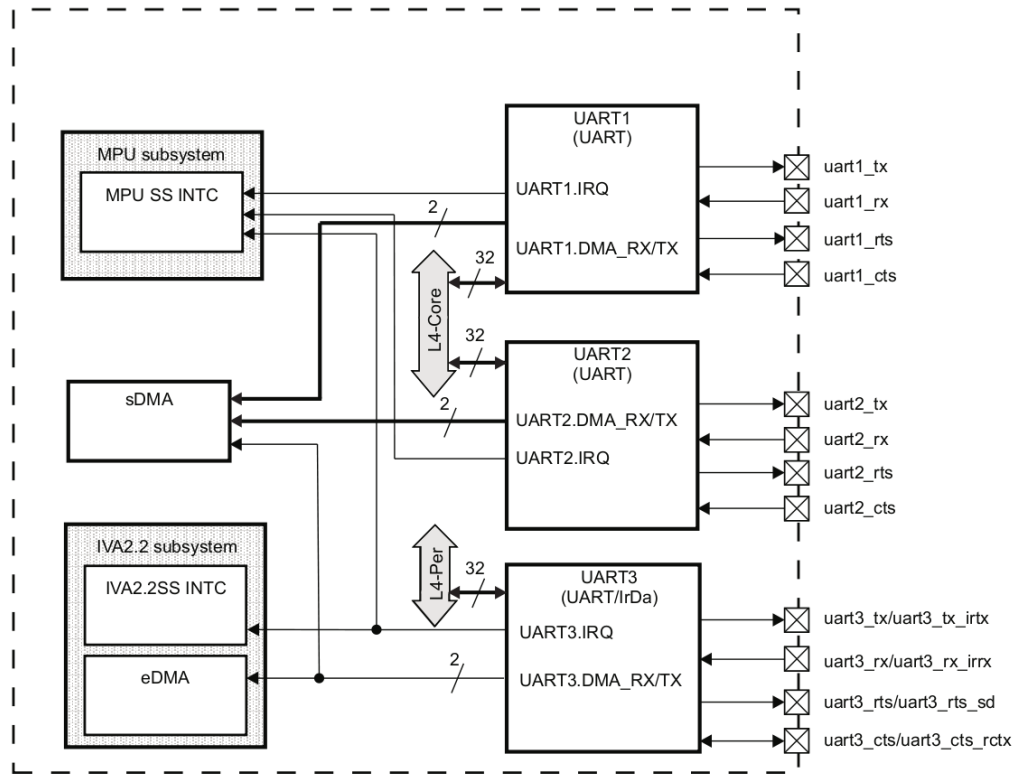


Fig 5.1 Blockdiagram UART

- the protocol of the UART-mode fix mostly a 8 bit transfer, 1 start bit and 1 or 2 stop bits.
- the protocol has to be the same for the transmit- and the receive-interface (no opportunity to check the protocol directly if the transmission failed).
- both the transmit- and the receive-interface can set flags and can generate interrupts if a transfer is finished.
- no error-check methods in the UART-mode (only parity-checking is possible).

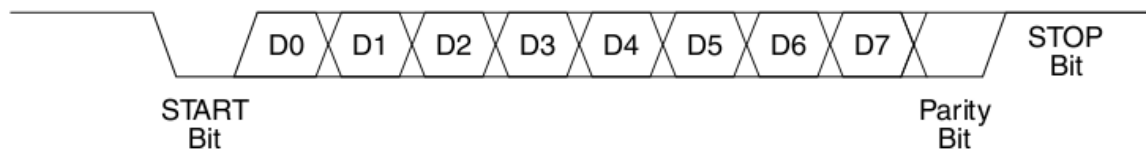


Fig 5.2 Protocol of a 8 Bit UART-Transfer

- the SIR-mode is similar to the UART-mode.
- every 0-bit generates an IR-pulse of a fixed duration (depends on the baudrate).

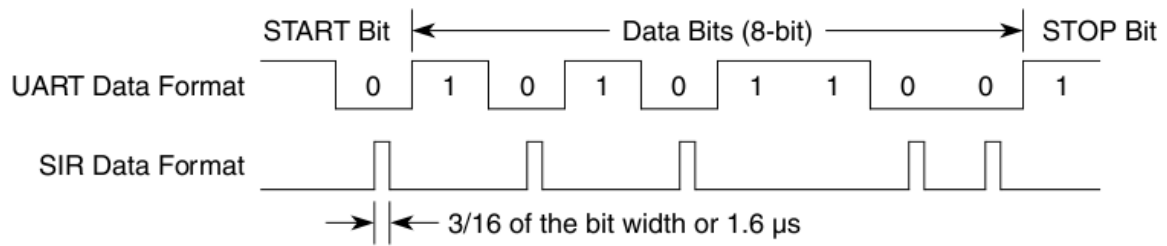


Fig 5.3 Protocol of a SIR-Transfer

- the MIR-mode has the same bit-coding as the SIR-mode, but with shorter IR-pulses (higher baudrates).
- in this mode the data is transferd in frames (HDLC-Frames)with a start-of-frame and stop-of-frame sequence (STA, STO).
- the frames are checked with an error check mechanism (CRC).

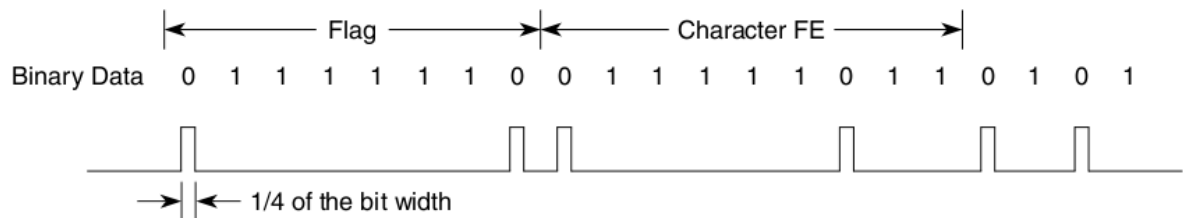


Fig 5.4 Protocol of a MIR-Transfer

- the FIR-mode use 4 PPM (4 pulse position modulation), each IR-pulse has a duration of 125 ns.
- the data transfer is done in frames.
- the frames are checked with an error check mechanism (CRC).



Fig 5.5 Protocol of a FIR-Transfer

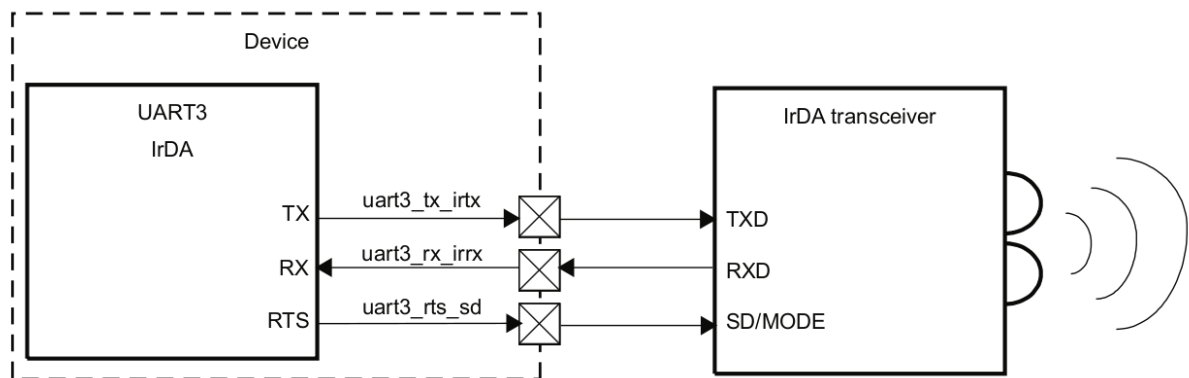


Fig 5.6 Example for an IrDA-Interface

USB (universal serial bus)

- supports the high-speed operation up to 480 Mbps and full-speed operation at 12 Mbps (USB 2.0 Specification).
- burst-transfer, interrupt and isochronous transport modes.
- 4 kbytes of shared FIFO RAM and 1 kbyte descriptor RAM.

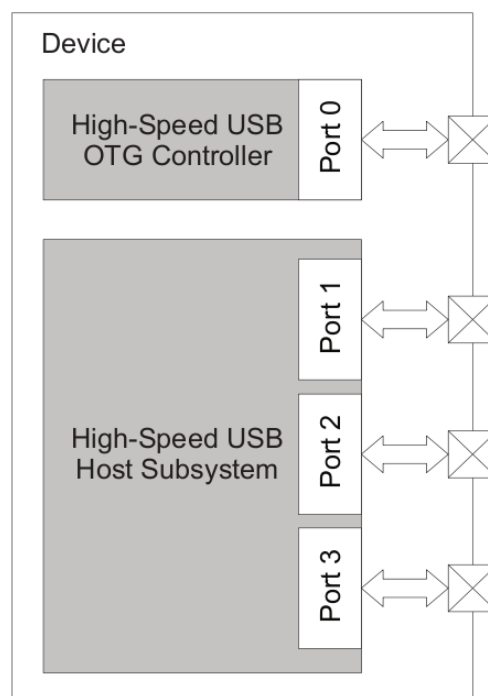


Fig 5.7 Blockdiagram USB

I²C (inter-integrated circuit communication)

- an interface to connect hardware decices (e.g. LCD-Displays, keyboards, switches).
- multi-master operation.
- different serial clock frequencies are programmable.

- interrupt driven byte-by-byte data transfer possible.
- 3 different I²C-Interfaces

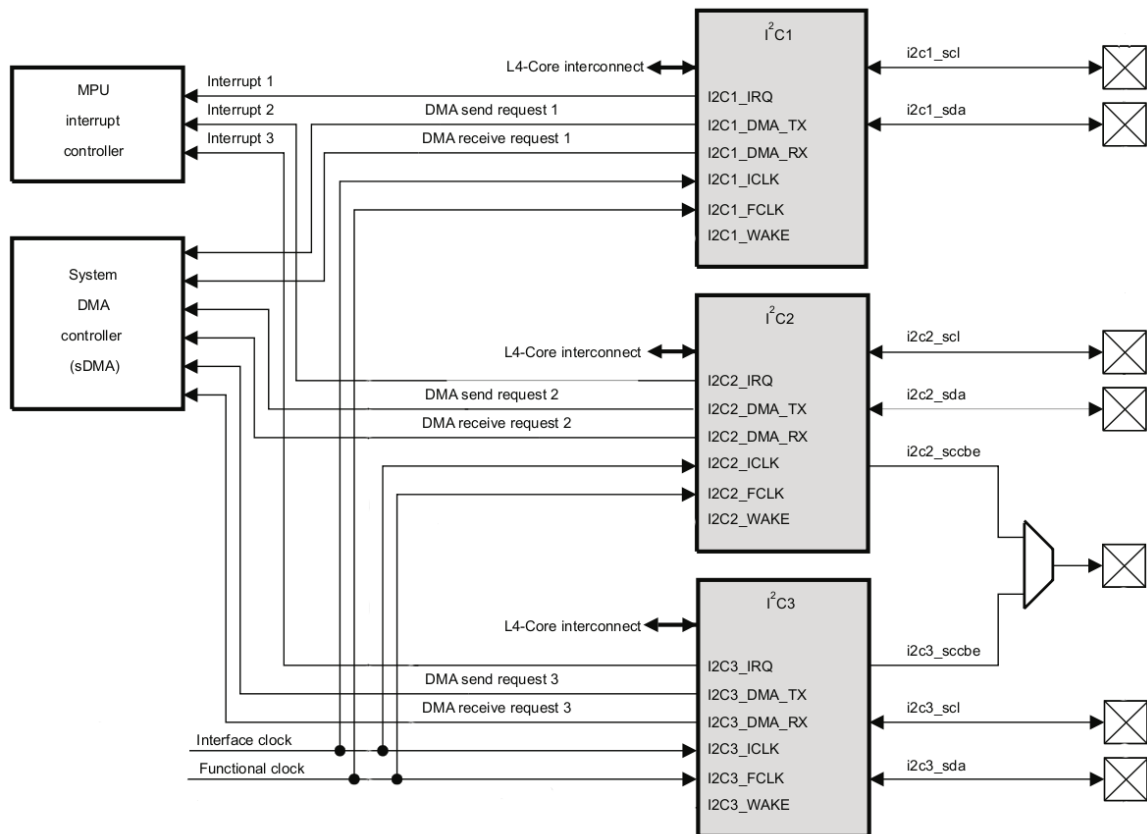


Fig 5.8 I²C-Blockdiagram

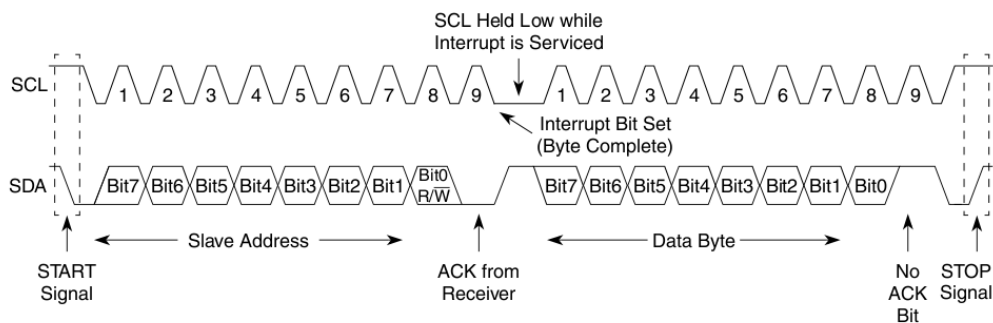


Fig 5.9 Typical Data Transfer Protocol of a I²C-Interface

5.4 Timers

GPT (general purpose timer)

- implementation of 11 independent timer channels
- every timer has a wordlength of 32 bit and operates as a free running timer.
- the timer channels could operate in:
 - an input capture-mode (in this mode the counters run until the specified capture event occurs (rise, fall, or pulse) on appropriate Event_Capture_IN-signal)

- the capture value are stored in a specific TCAR-register and could be read
- an output compare-mode (in this mode the counters run until they reach the programmed terminal count value. At this point, the specified output event is generated (toggle, pulse high, or pulse low) on PWM_OUT)
- the compare value are stored in a specific TMAR-register
- a PWM-mode (in this mode the user can program period and width values to create an adjustable, repeating output waveform on PWM_OUT)
- PWM-mode: suitable to emulate an analogue output signal (with an additional operational amplifier).

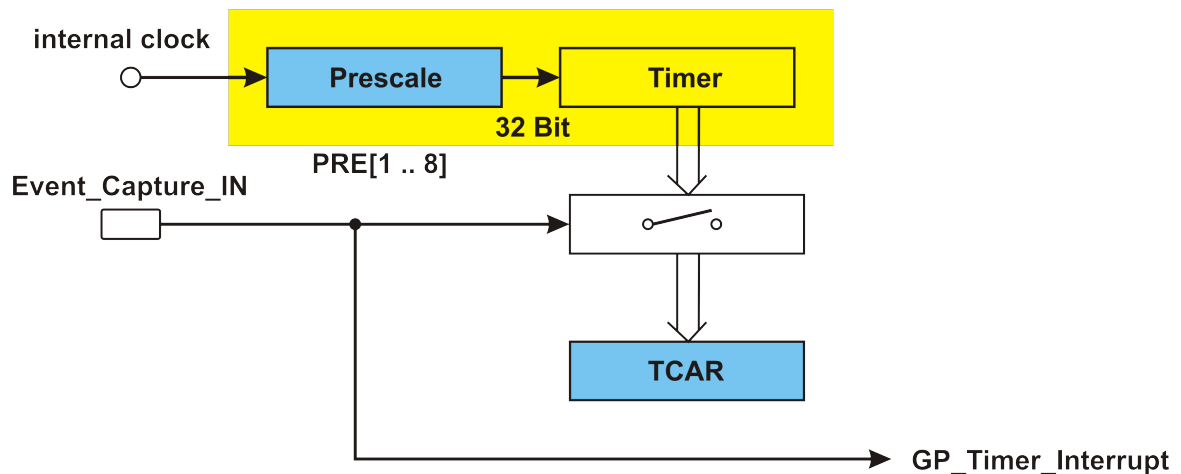


Fig 5.10 Blockdiagram of the Input Capture Mode

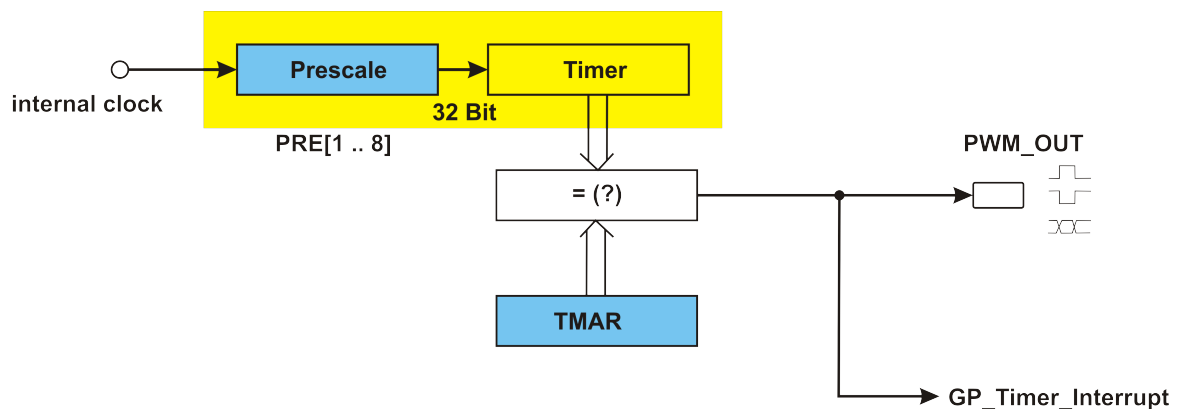


Fig 5.11 Blockdiagram of the Output Compare Mode

Watchdog Timers

- two different timers with a specific clock signal (32 KHz)
- each timer consists of a 32-bit counter with an additional prescale (1 .. 128)
- the timers generate a special reset (watchdog) and a special interrupt signal (if not reloaded in time)
 - after they have reached zero, the timers reload and start the counting procedure one's more.

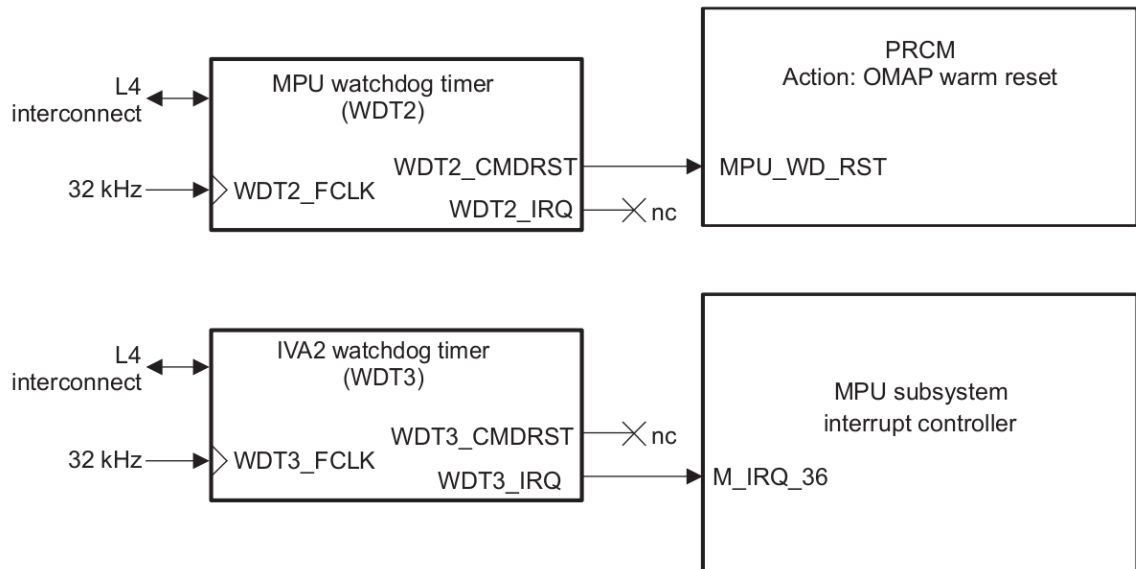


Fig 5.12 Blockdiagram Watchdog Timers

Chapter 6

Operating Systems

6.1 Operating Systems for Embedded Systems

Advances in microcontroller technology continue to open up new application fields for embedded systems. In parallel with this trend, the equipment controlled by embedded systems has become more sophisticated, often incorporating many functions in one product. As a result, embedded systems have grown in scale and complexity. Moreover, as products increasingly adopt digital technology, advanced processors have enabled more of the processing to be implemented in software, making embedded systems all the more important.

This means, in new application the developer is more and more interested to do the software development with the support of an operating system. Especially in large scale systems the effect on reducing development cost with an operating system is important.

Even in the small-scale embedded system field, the growing scale and complexity of software and the need for fast development turnaround time have made improving software productivity a pressing need. The use of C and other high-level languages have become increasingly common for this reason.

Real time operating systems fulfil all the requirements from the developer. They guarantee a response of the system in specific time. The development cycle becomes much shorter if the user could do the development on the target. Cross-development involves the downloading of the executable program on the target, testing it, changing it and repeating the same procedure once more for the next error is not an optimal development cycle. Real time operating systems offer a good support for the debugging.

There exist only a few drawbacks if you are looking for appropriate real time operating system. The first most important disadvantages of the most real time operating systems are the costs. The second one is that real time operating systems need virtual memory system which is in special not available in small microcontrollers.

The virtual memory system itself is really suitable for general purpose applications like a desktop PC. But for embedded systems a virtual memory system makes the application more difficult. In such systems really often an access to the hardware is necessary (control interfaces). These programming aspects could only be done in the so called “kernel-mode” with the result that such programs more dangerous (collapse of the operating system) and more difficult to test.

In this chapter we will discuss some aspects of operating systems and than come to the uCLinux or other “embedded Linux Systems”, operating systems for microcontroller with and without a virtual memory system.

- a lot of embedded systems runs without a operating system
 - only small algorithm (e.g. 'control the window' open/close system in a car).

- hard restrictions in the size of memory especially by 8- and 16-bit microcontrollers.
- costs of an operating system.
- reasons for the use of an operating system
 - generally the same reasons why we need one for a traditional computer.
 - but in the case of embedded systems not all services of an operating systems are needed for any device.
 - embedded systems have a large variety of requirements and environments.
 - critical applications with high functionality (medical applications, aircraft systems ..).
 - critical applications with small functionality (ABS, cardiac pacemaker ..).
 - not very critical applications with varying functionality (mobile phone microwave oven, washing machine ..).

Special operating system (OS) for embedded systems

- a standard OS (like Windows or Linux) isn't suitable because :
 - monolithic kernel has too many features which are not necessary in a specific application.
 - monolithic kernel is not modular, fault-tolerant, configurable, modifiable.
 - needs too much memory space (virtual memory).
 - not designed for mission-critical applications.

Key features of an OS for embedded systems

- configurability → no single OS will fit all needs, so configurability needed
 - simplest form → remove unused functions (by linker ?)
 - conditional compilation (using #if and #ifdef commands)
 - dynamic data might be replaced by static data
 - removes unnecessary functions or modules by a kernel programming (Linux, eCos, VxWORKS).
- handling of device drivers
 - access to interfaces.
 - device drivers handled by tasks instead of integrated drivers.
 - improve predictability; everything goes through scheduler.
 - in embedded systems mostly a lot of different interfaces are available, so not every device needs support by all versions of the OS.

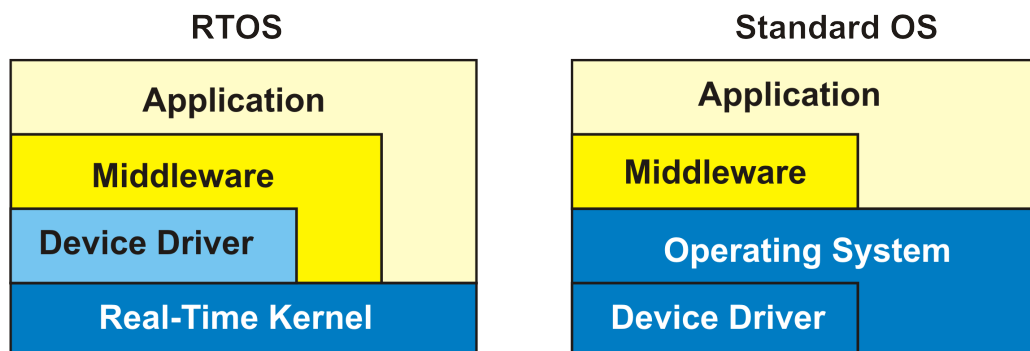


Fig 6.1 Integration of Devices Drivers in an OS

- handling of interrupts
 - for standard OS: serious source of unreliability.
 - it should be possible to let interrupts directly start or stop tasks (by storing the tasks start address in the interrupt table).

Available OS for embedded systems

- QNX
 - real time operating system based on micro core technology.
 - suitable for small and large embedded systems.
 - tasks are separated in its one address space.
 - scheduling event driven (restrictions in time based scheduling) .
- VxWORKS
 - small VxWorks Kernel.
 - no virtual memory space (dangerous if a task is not cooperative).
 - scheduling time, priority and event driven.
 - implemented on a lot of microcontrollers.
- OSEK/VDX
 - scalable real time OS build for application in the automotive area.
 - event driven execution of tasks.
 - operates with fixed priorities.
 - tasks are grouped in basic tasks and extended tasks.
 - no virtual memory system.
- eCos
 - open source system.
 - scalable system (the configuration allows the application writer to impose their requirements on the run-time component).
 - OS and application software are specific for an embedded system.
 - scheduling is time and event driven.
 - no virtual memory system.
- embedded Linux
 - open source system.
 - scalable and portable (especially for 32 bit microcontrollers).
 - run with a non virtual and a virtual memory system.
 - scheduling is time and priority driven.
 - with the RTAI (real time application interface) also a event driven scheduling mechanism possible.
 - for small and large embedded systems.

6.2 Functions of Embedded Linux

- the general idea of an “embedded Linux” is to store an image (Kernel plus necessary modules) in a non-volatile memory (Flash-EEPROM, SD-Card).

- after a reset (or power on reset) the image is copied in the RAM and executed.
- the size of the copied image is also known as a “footprint”.
- the additional feature of embedded CLinux (Linux for microcontrollers) is has the opportunity to work without a virtual memory system (VM).
 - suitable for processors without MMU.
 - low memory footprint.
 - according to the processors low power consumption applications are possible.
- with the ARM-Processors (MMU is available) embedded Linux could run with a virtual memory system or with a non virtual memory system.

Boot sequence

- the boot procedure of embedded Linux differs from a standard boot procedure.
- this procedure offers two advantages:
 - the development process of building a Linux system is nearly the same as in a standard Linux (the same tools are available).
 - the execution time of programs which runs in a RAM is much more faster than in a Flash-EEPROM).
- boot procedure starts immediately after the reset (an interaction with the user isn't necessary).

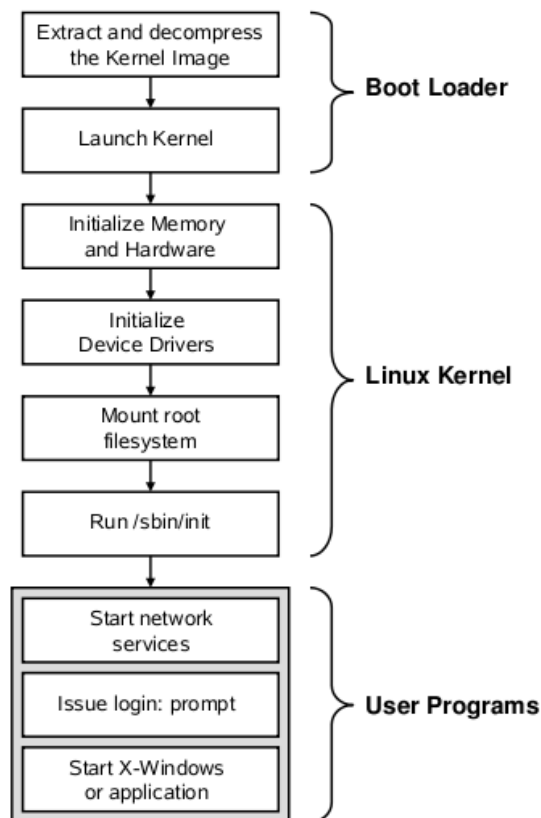


Fig 6.2 Boot Procedure

Scalable behaviour of embedded Linux

- building of a new image (binaries of embedded Linux systems) starts with a configuration part.
- first part is the selection of the “vendor”.
- specification of hardware details of the development kit (RAM addresses, ROM addresses).
- the next step is the configuration of necessary modules and drivers.

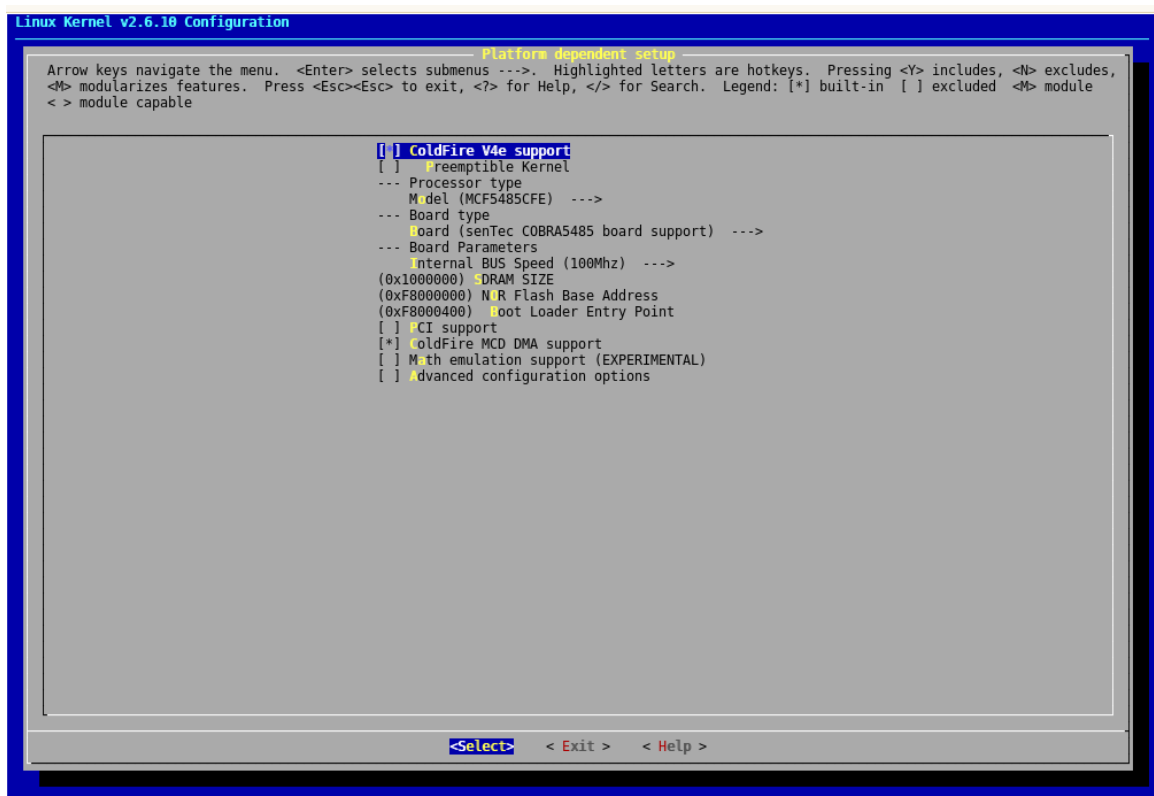


Fig 6.3 Specification of Hardware Details

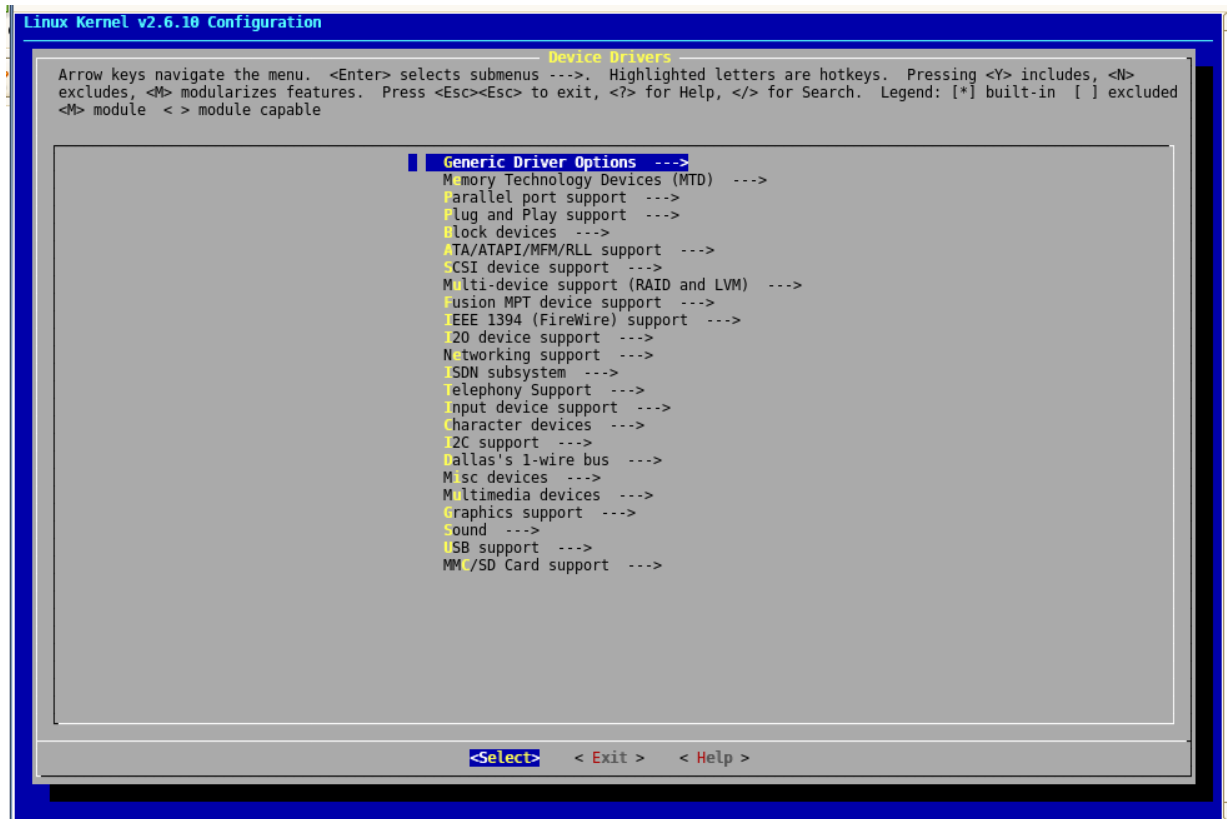


Fig 6.4 Configuration through a Selection of Moduls or Drivers

Build procedure

- after the configuration (make menuconfig)
 - start the build process with “make dep”.
 - and “make”.
- the Kernel-compilation need a special compiler (gcc-version for the ARM-Processor) und a special binutils (as, ld objtools for the ARM-Processor).
- the result of the Kernel-compilation is a image which is available in two versions.
 - file “linux.bin or Kernel.img” .
 - a loadable image.
 - could be downloaded (serial or usb-interface, needs a monitor program on the target side).
 - execute after a poweron reset or with a jump on a specific address (monitor function).
 - useful for test of a configuration.
 - file “romfs”
 - a image which could be loaded in the FlashEPROM.
 - need a “Flash”-load program like CF-Flasher.
 - embedded Linux starts after the next reset or power-on-reset with the mentioned boot sequence.

Working with embedded Linux in the target

- establish a serial communication link between the target and the PC.

- works as standard input/output (stdio) for messages from embedded Linux.
- on the PC-side, a terminal program like “minicom” for Linux or ”Hyperterminal” for Windows is useful.

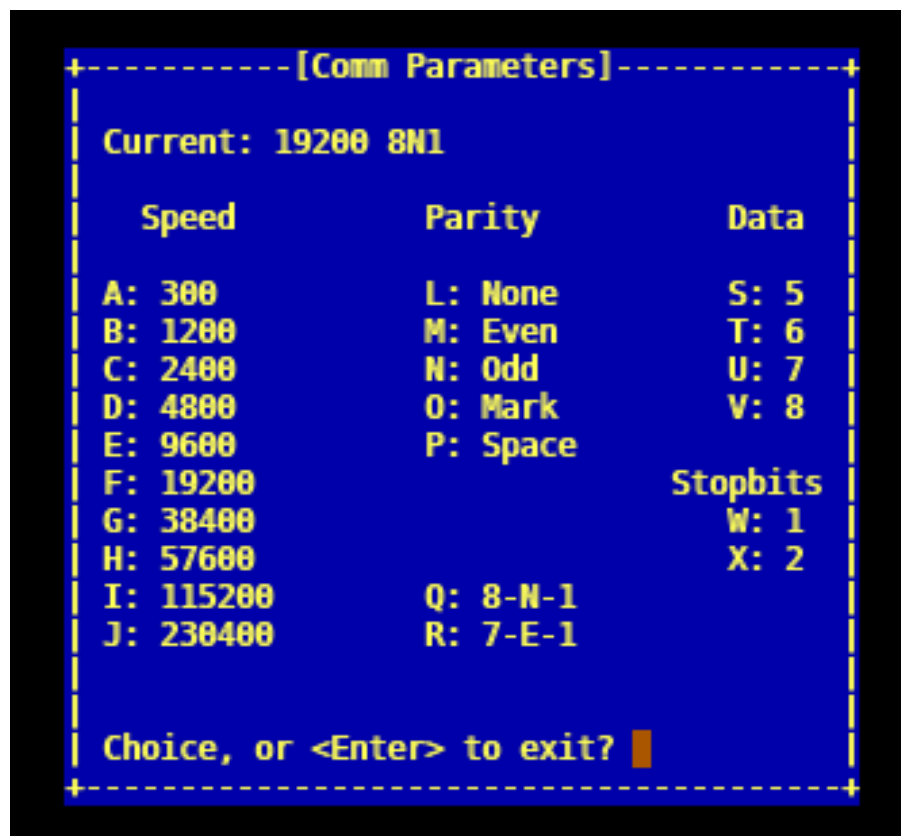
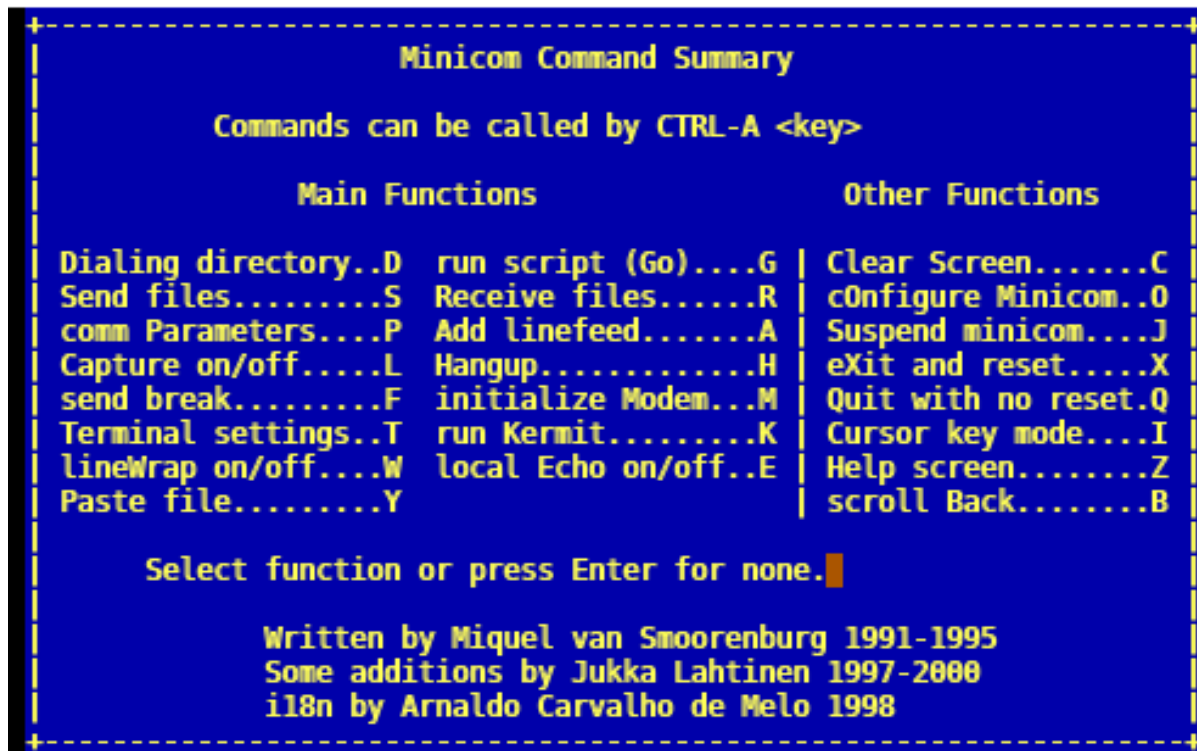


Fig 6.5 Configuration of Minicom

- if an ethernet link is established
 - generate a remote connection (via ssh)
 - execution of commandline instructions on the target
 - mount PC partition by NFS
 - target-filesystem can be mounted in the development-filesystem
 - execute programs on the development-side and on the target-side

```

sh-2.05# pwd
/bin
sh-2.05# ls
addgroup  cpio      fgrep     iptunnel  mt        sed        umount
adduser   date      getopt    kill      mv        sh         uname
ash        dd        grep      ln        netstat   sleep     usleep
bash      delgroup gunzip    login     pidof     stty      vi
busybox   deluser   gzip      ls        ping      su         watch
cat        df        hostname  mkdir     ps        sync      zcat
chgrp     dmesg     ipaddr    mknod     pwd       tar
chmod     echo      ipcalc    mktemp    rm         tinylogin
chown     egrep     iplink    more      rmdir     touch
cp        false     iproute   mount     run-parts true
sh-2.05#
  
```

Fig 6.6 Standard Commandline Interface on the Target-side

Restrictions when using non VM-systems

- ARM-Processors could operate with a VM (virtual memory) and a non VM System.
- in the case of working without a VM-System:
 - a user program has full access to the whole memory space.
 - as a result, no segmentation fault is possible.
 - no dynamic heap → instead uCLinux uses a global memory pool (kernel free memory pool).
 - no obvious implementation of malloc due to lack of heap.
 - no fork system call → an implementation of “copy on write” strategy isn’t possible (an alternative is the old BSD vfork and all its limits).

6.3 Cross Compilation

- cross compilation is widely used for programming in embedded systems.
- the programs are written, linked and simulated on a PC (for example) for an embedded system.
- then these executable programs have to be downloaded and started on the target under the control of a monitor program (mostly with a serial connection between the PC and the target).
- after the test in the target the executable program could be stored in the EPROM and executed without a monitoring program

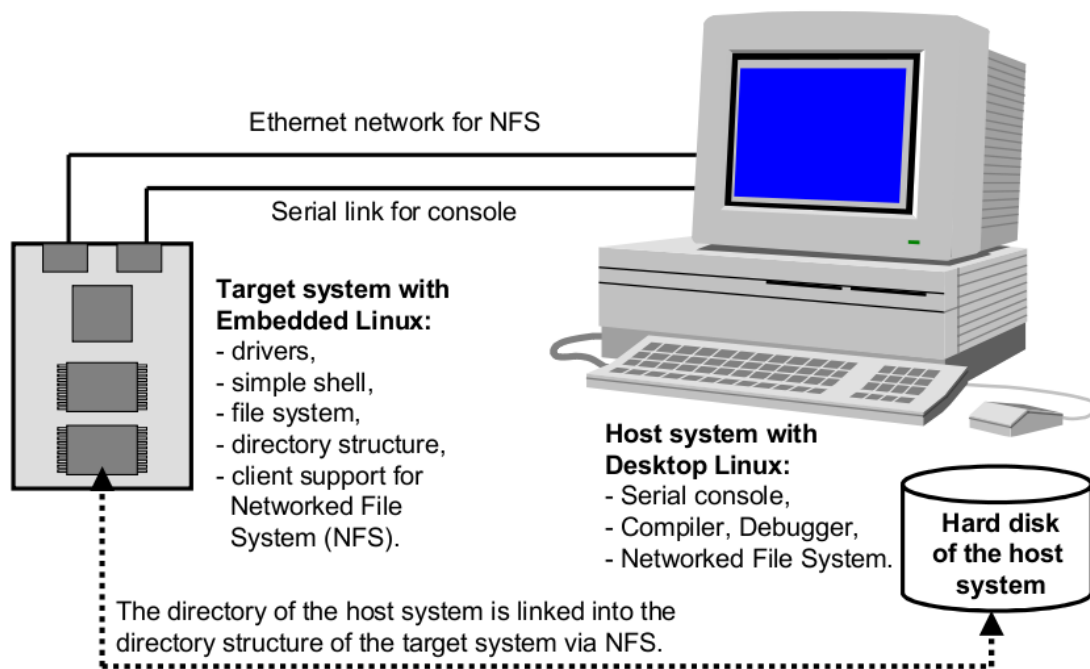


Fig 6.7 Cross Compilation

Cross compilation with embedded Linux

- in embedded Linux systems the cross compilation is done for the same reasons as for a standard system.
- the advantage in this case is that the same debugging tools that are available for the test on the PC are also available on tests on the target system (remote debugging).
- to compile a program for the ARM-Processor a specific toolchain is necessary
- the same toolchain is also used to build the embedded Linux System for the target.
- to generate all parts of the toolchain the standard C-compiler and C-libraries are also involved:

Tool	Description
arm-linux-gcc	C-Compiler for ARM-Processor
arm-linux-as	Assembler for ARM-Processor
arm-linux-ld	Linker for ARM-Processor
arm-linux-ar	Archiver, tool to build, extract and show the contents of libraries
arm-linux-nm	extracts the symbols of an executable file
arm-linux-objdump	extract the contents of an executable file (e.g. disassemble files)
arm-linux-objcopy	tool to generate loadable files from the output of the linker (e.g. "x.bin" or "x.ihex")

Executable files

- standard output of the linker ("x.elf") isn't downloadable.

- this file needs a VM-System for loading and execution.
- instead of “x.elf” the tool arm-linux-objcopy generates a “x.bin” and/or “x.ihex”-file.
 - both files are directly downloadable and don’t need a VM-System for execution.
 - this means both file are linked on an absolute address.
 - the “x.bin”-format is a binary format (could not be read with an editor).
 - the “x.ihex”.format is a text format which is readable with any standard editor.

```

5/DBG_Led/Led.elf
ELF Header:
  Magic:   7f 45 4c 46 01 02 01 00 00 00 00 00 00 00 00
  Class:                   ELF32
  Data:                     2's complement, big endian
  Version:                  1 (current)
  OS/ABI:                   UNIX - System V
  ABI Version:              0
  Type:                     EXEC (Executable file)
  Machine:                  MC68000
  Version:                  0x1
  Entry point address:      0x20000
  Start of program headers: 52 (bytes into file)
  Start of section headers: 9112 (bytes into file)
  Flags:                    0x0
  Size of this header:      52 (bytes)
  Size of program headers:  32 (bytes)
  Number of program headers: 3
  Size of section headers:  40 (bytes)
  Number of section headers: 15
  Section header string table index: 12
bm@linux-04z2:/usr/local/m68k-elf/LinuxBSP_20061018/toolchain/m68k-linux/gcc-3.4.0-glibc-2.3.2/bin>

```

Fig 6.8 Information in a “x.elf”-file

6.4 User Programming in uCLinux

- user programming implies that a program runs in the user mode of the ARM-Processor.
- if the VM-System is activated → the user doesn’t have an access to an absolute memory address.
 - especially a direct access to the registers of interfaces isn’t allowed.
- the restrictions imply also that an access to R-, S-, D-registers, CPSR and etc. are not allowed.
- these access could only be done in the supervisor mode (kernel program).
- Development of program should be done in tree steps:
 - first try to generate a program with a cross-compilation for the target, load and then test it.
 - then merge the program together with an image of the operating system and load it.
 - lastly, if necessary implement a ”autoexec” function in the init process of the embedded Linux.

Cross-compilation with embedded Linux

- the cross-compilation with embedded Linux has the same procedure as the standard cross-compilation for ARM-Processors.
- in preparation for the next two steps it is helpful to use a separate directory which holds all the necessary files (C-files, Header-files, makefile).
- put this directory in the ARM-Linux-file-system '/armLinux/user/' .
- after the compilation mount the directory with NFS in the uCLinux for the target and execute and test the program.

Download together with an image of embedded Linux

- the second step is to integrate the program in an image of uCLinux.
- additional to the cross-compilation, the program also has to be integrated in the build procedure of uCLinux.
 - the “central”-makefile for building all the user applications (/armLinux/user/makefile) should be expanded with an instruction like:


```
DIRSc$(CONFIG_USER_CAN)      +=  can4linux  (present line)
DIRSc$(CONFIG_USER_PROGRAM) +=  program    (new line)
```
 - to make the program visible in configuration tool (make menuconfig) of arm-Linux the file 'config.in' (/armLinux/user/config.in) should be expanded by:


```
bool 'can4linux' CONFIG_USER_CAN      (present line)
bool 'program'   CONFIG_USER_PROGRAM (new line)
```
 - to get some more information about the program and how somebody can do the configuration the file Configure.help (/armLinux/user/Documentation/Configure.help) should be expanded by


```
CONFIG_USER_CAN
```

This package contains some examples
using can4linux device driver.

CONFIG_USER_PROGRAM

This program adds my first functions
to uCLinux. I'm not sure whether
somebody can use these functions.

- if in the configuration step, the program is selected, the resulting image contains the program
- to test the image (armLinux) and the program once more, it can be downloaded with the monitor
- the downloaded armLinux (including the program) could be started after a re-boot or a reset

Autostart of a program with embedded Linux

- if the program (which was downloaded together with the armLinux-image) has to be used, the user has to start it.
- embedded Linux offers an automatic start of programs (function of the program init which executes the script file /etc/inittab).
- the script-file inittab asks another script-file rcS (placed in /etc/rc.d/rcS) and this file switches over to rc.local (/etc/rc.d/rc.local).

- an auto start could be done by addition in the file rc.local (but this means that the user has to change this file and this is obviously not an autostart, especially not for the first time).
- to do a 'real' auto start of the file
/armLinux/defconfigs/vendors/senTec/etc/rc.d/rc.local has to be changed and then a new image has to be created.
- when this image is loaded in the FlashEPROM (with the program CFFlusher) then after a reset the program is automatically started without an interaction with the user.

Bibliography

- [1] Wikipedia – The free Encyclopedia. www.wikipedia.org.
- [2] B. P. Douglass, *Real-Time UML, Second Edition*. Addison Wesley Longman, Inc., 2000. Reference for Real-Time-Modulation.
- [3] P. Marwedel, *Embedded System Design*. Springer Verlag, 2006. A good Overview about Embedded Systems.
- [4] D. Abbott, *Linux for Embedded and Real-time Applications*. Elsevier Science, 2003. Implementation of Embedded Linux.
- [5] *Reference Manuals*. <http://www.freescale.com/>